System Verification and Validation Plan for Software Engineering

Team #14, Troitsky Toolbox
Pedro Azevedo
Mason Azzopardi
Michael Roberts
Jack Easton

November 1, 2026

Revision History

Date	Version	Notes
October 27, 2026	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
  - 2.2.1 Primary Objectives . . . 2
  - 2.2.2 Out-of-Scope Objectives . . . 2
  - 2.2.3 Assumptions . . . 3
- 2.3 Custom Documents . . . 3
  - 2.3.1 Fall - Interviews . . . 3
  - 2.3.2 Winter - Usability Report . . . 3
- 2.4 Relevant Documentation . . . 4

3 Plan **4**
- 3.1 Verification and Validation Team . . . 5
- 3.2 SRS Verification . . . 6
- 3.3 Design Verification . . . 8
- 3.4 Verification and Validation Plan Verification . . . 8
- 3.5 Implementation Verification . . . 9
- 3.6 Automated Testing and Verification Tools . . . 10
- 3.7 Software Validation . . . 11

4 System Tests **12**
- 4.1 Tests for Functional Requirements . . . 12
  - 4.1.1 Area of Testing1 . . . 13
  - 4.1.2 Area of Testing2 . . . 14
- 4.2 Tests for Nonfunctional Requirements . . . 14
  - 4.2.1 Area of Testing1 . . . 14
  - 4.2.2 Area of Testing2 . . . 15
- 4.3 Traceability Between Test Cases and Requirements . . . 15

5 Unit Test Description **15**
- 5.1 Unit Testing Scope . . . 15
- 5.2 Tests for Functional Requirements . . . 16
  - 5.2.1 Bridge Failure Module . . . 17
  - 5.2.2 Rules Compliance Module . . . 19

5.2.3 Bridge Scoring Module 22
5.2.4 Log-in 24
5.2.5 Materials Testing 25
5.2.6 Knowledge Repository 27
5.2.7 Knowledge Transfer 28
5.2.8 Toolbox Broker 29
5.3 Tests for Nonfunctional Requirements 30
5.3.1 Module Analysis 30
5.4 Traceability Between Test Cases and Modules 31

6 Appendix **32**
6.1 Symbolic Parameters 32
6.2 Usability Survey Questions? 32

7 Appendix — Generative AI Use Disclosure **33**
7.1 AI Tools Used 33
7.2 How and Where Used 33
7.3 Verification of AI-Generated Content 33

8 Appendix — Reflection **34**

List of Tables

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

The Troitsky Toolbox project seeks to create a centralized information repository and a collection of software services that provide technical support for the McMaster Troitsky club. These services ensure information transfer between years and teams, and facilitate support and onboarding for new members. Served as a web application, accessible by all current club members, the services provided will simplify the technical components of the bridge design process, decreasing the barrier to becoming a competitive team.

One of the primary services in the toolbox is the model analysis service, which allows teams to load STEP files of their bridge designs and perform basic analysis on the models. This service will provide teams with a simplified analysis of their design, when compared to external static model analysis software, and will integrate with the material testing component and the rule compliance checker to provide a more specified analysis given the build material properties and current competition rules. Services regarding the information repository will include the ability to search through past designs and load them into the model analysis service, allowing teams to upload their designs with exit documentation and providing customized material testing result upload features. A knowledge transfer service collects executive handoff documents so that club knowledge carries over between years. In managing the toolbox access, role-based access control (RBAC) will be integrated to ensure that information is kept within the club, limiting access, and allowing executive members the ability to manage club users.

2.2 Objectives

The primary objective of the verification and validation plan is to outline the approach to ensure that the Troitsky Toolbox project will meet the requirements outlined in the software requirements specification. This plan will help to build confidence in the correctness, reliability and safety of the system, focusing verification on high-risk points and safety-critical concerns outlined

in the hazard analysis.

The V&V plan will primarily focus on these system objectives outlined below. There are also goals outlined that have been deemed out of scope for the current development cycle, but will be outlined in documentation as goals for future year development.

2.2.1 Primary Objectives

- **Adequate model analysis correctness:** Testing the correctness of the model analysis service is vital to the usability of the toolbox, as, without ensured correctness, teams will not be able to rely on the results. These results will not only be used by teams in the bridge design process, but will also guide them in making estimates of their bridge's weight and ultimate load capacity.

- **Data integrity and persistence:** The system must ensure data integrity and security in the knowledge repository. The repository will serve as the club's main data storage and information transfer, so losing data would be a critical concern and would defeat the project's purpose.

- **Access control:** The RBAC in the club will maintain data privacy both between teams and outside of the club. Testing this access control will provide confidence to teams when storing current bridge designs in the system, and ensure that executive and exit documentation is private to the club.

- **Demonstrating functional usability:** The functional usability of the system determines if it will be fully integrated into the club's organization and individual McMaster teams' bridge design and development process.

2.2.2 Out-of-Scope Objectives

- **Industry standard static model analysis performance:** The model analysis service is intentionally designed to provide a simplified approximation of static model analysis, so its accuracy will have a larger tolerance than industry-standard tools and will not aim to match established services.

- **Full year competition testing:** The toolbox testing will not include an end-to-end full-year competition evaluation within the current development cycle. This would require a usable product from the beginning of the school term, allowing teams to use the service throughout their full design process.

2.2.3 Assumptions

There is an assumption of the correctness of external libraries (BFE.NET, IxMilia.Step, occt-import-js, three.js, SQL Server) used. It is assumed that they are verified by their maintainers; only our integration with them is tested.

2.3 Custom Documents

2.3.1 Fall - Interviews

The primary CD for the fall term development is interviews conducted with core stakeholders: experienced Troitsky club members and executives. These semi-structured interviews will help outline expectations and prioritization of services from knowledgeable members. These interviews will be anonymized to ensure user privacy and aggregated into usable results for the toolbox development process. The interviews will be vital in checking any assumptions made in the SRS with experienced club members.

In addition to the core interviews for the fall CD, a survey has been performed to begin the requirements elicitation process. The survey design record and raw data are stored in the project repository, and the results were used in the SRS (Azevedo et al., 2026b) to outline the priority of services and core issues members are currently facing.

2.3.2 Winter - Usability Report

The CD completed in the winter term will be a usability report, allowing for a more complete assessment of the functional usability of the toolbox. This round of testing will be performed with a more complete iteration of the product, allowing for more complete analysis of different system services' usability. Users will be gathered spanning different experience levels, both in club experience and technical experience (modelling), creating valuable data about the system's performance. Specific user functions will be tested, with

tasks that follow the G.5 usage scenarios, measured by completion rate and time on task.

This CD will further strengthen the V&V of the system, ensuring that complete functional and non-functional requirements come together into a fully usable service.

2.4 Relevant Documentation

1. **Software Requirements Specification (Azevedo et al., 2026b):** The SRS defines the functional and non-functional requirements for the Troitsky Toolbox, and is referenced in the V&V plan as the standard the system wishes to meet. The requirements set the goals for the testing areas, identifying the importance of different services, which is then reflected in the testing performed for those given systems.

2. **Hazard Analysis (Azevedo et al., 2026a):** The hazard analysis provides information regarding the primary risks and failure points of the system, pointing to the components that will require more in-depth and complex system testing. The hazard analysis also defines safety requirements for the system which are referenced in the V&V plan specifically with regard to system safety and access control correctness.

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

This sections contains the grand plan to verify and validate this project's documents and implemenation. It outlines the team, how every document, including this on, will be verified, how the implemenation will be verified, automated testing technology that will be used, how assumptions will be verified, and how contents, both internal and external will be validated.

Artificial intelligence (AI) shall be liberally employed to help us brainstorm additional ways to verify and validate all documents. However, AI frequently hallucinates and thus cannot validate or verify anything itself.

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

This project is immense and requires a lot of work to verify and validate. Peer review will be a guiding principle to catch individual errors and therefore, peer subject matter expertise shall be used instead of quality assurer and subject matter expertised being fused into one person. The following table shows the division of labour:

Name	Project Role	VnV Responsibilities
Mason Azzopardi	Project Supervisor, Back-end Developer	As project supervisor, he shall oversee all the work and guide the team to success. As a back-end developer, he shall serve as the secondary back-end expert to aid VnV related to the back-end.
Pedro Azevedo	Troitsky Liason, Front-end Developer	As Troitsky liason, he shall be the primary point of contact between this team and members of the Troitsky club. As front-end developer, he shall serve as the front-end expert to VnV related to the front-end.
Jack Easton	Back-end Developer	Serve as the primary back-end expert to aid VnV related to the back-end.
Melvin Mokhtari	Teaching Assistant	Answer administrative questions and provide feedback on course expectations and grading.
Michael Roberts	Quality Assurer	Spearhead VnV efforts. Consult with subject matter experts to draw on their expertise. Develop a robust testing suite to verify the project and consult with Troitsky Club members, especially the club president, to validate the project.
Isabella Vescio	Supervisor	An *ex officio* role stemming from her role as Troitsky Club President, she shall provide feedback so the project fits the needs and desires of the Troitsky Club. She shall involve other club members in the process as needed to assist her in this task.

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the

review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

The Software Requirements Specifications shall be verified in the following ways. In case mistakes are found or improvements are suggested, as they inevitably will, corrections will be made and the corrections shall be reviewed again. The above process is an iterative process and shall only terminate when the portion is deemed satisfactory.

- **Checklist:** Two checklists shall be developed by the Quality Assurer to verify the requirements, one for each requirement and one for the document as a whole. The former checklist shall be explicitly evaluated on each requirement to determine its quality, and will include qualities like unambiguity, verfiability, traceability, and correct spelling and grammar. The latter shall cover the pitfalls a granular analysis cannot cover, including internal consistency, completeness and documented modifiability.

- **Internal Peer Review:** Each portion of the document, especially each requirement shall be reviewed by a different group member than its author. The whole document shall be reviewed through a group-wide read-through of the document at a meeting.

- **Troitsky Review:** The Troitsky Club is our client and its members are frequently available in order for us to seek their feedback. While showing them the SRS is a fool's errand, all assumptions baked into the document shall be verified by club members through asking if the assumption is true in non-technical language they can easily understand.

- **External Peer Review:** Other capstone groups will raise issues regarding our SRS through GitHub issues. We shall address those issues, and either incorporate their feedback or dismiss it after determining their suggestion is wrong or not necessary.

- **Instructional Review:** The instructional team, primarily our assigned Teaching Assistant, will look over, comment upon, and grade the SRS. We shall seek to incorporate their feedback into our work.

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS] [Create a checklists? —SS]

The design shall stem from the requirements and shall be verified in the ways below. The verification shall be an iterative process and shall continue until the design is determined to fulfill the needs of the project.

- **Unified Modelling Language Diagrams:** All designs shall be converted from text into corresponding UML diagrams, which will facilitate verification. Many errors will be caught and fixed at the translation stage.
- **Internal Peer Review:** All designs and associated commentary shall be carefully reviewed by a different group member than the design and commentary writer. Feasability, proper decomposition, and other design principles (especially the SOLID principles) shall be guiding lights. The full list shall be contained in a checklist that the Quality Assurer will develop.
- **Assumption Verification:** All newly-encountered assumptions shall not be silently implemented into the design. Rather, they shall be verified by the Troitsky club.
- **External Review:** The group and TA assigned to peer review us shall review our design after submission and provide feedback. Furthermore, meetings with the TA will be utilized to gain more suggestions.

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

This document shall be verified in the ways outlined below. All errors and weaknesses discovered through this iterative process shall be fixed and improved, respectively. This process shall only terminate when all errors are fixed and all weaknesses are either fixed or shown to be infeasible to fix.

- **Internal Peer Review:** All sections shall be reviewed by a different team member than its author.
- **Mutation Testing:** To prevent a lazy approval, authors shall deliberately shall write at least one error in their section. Therefore, if the peer is lazy and approves a section without checking it, the author can adjure the reviewer to review the section again.
- **Assumption Verification:** Any new assumption that is absent from previous documents shall be verified by the Troitsky Club, lest the assumption is wrong.
- **Final Group Read-Through:** Before submission, this document shall be read through in a group meeting to catch holistic errors, such as contradiction, spelling and grammar errors, missing links, and incompleteness.
- **External Review:** We will recieve feedback, including issues from the external peer reviewing group in addition to grades and comments from the marking TA.

While certain verification is impossible because any verification would require its own verification, near-certainty will suffice if this document is faithfully followed.

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

The implemenation shall be throughly tested to find syntactic errors, semantic errors and weaknesses to be strengthened. Due to our meticulous development plan, requirements document, and design documents, test-driven development (TDD) is an excellent choice. The Quality Assurer shall write

these tests, drawing on his colleagues' expertise for guidance. Testing the formalized portions is low-hanging fruit, and the remaining functions will be demecrated in the design process. Of course, TDD will not catch everything so additional unit tests will be necessary after the code is written to test more cases and fill in any omissions in the pre-written tests. The requirements shall play a paramount role in determining the content of the tests. To verify the fulfillment of the requirements, each functional requirement and each objective non-functional requirement shall have at least one test verifying it. The remaining non-functional requirements will be qualitatively verified through the Quality Assurer and the Troitsky Club.

The above covers unit testing. Once the unit tests are complete and have all passed, integration and system testing shall begin. When possible, they should be written with a unit testing framework, the only difference being the test tests multiple systems and is thus an integration test. However, much of integration testing and virtually all of system testing cannot be verified automatically and must be verified manually through the Quality Assurer running the program and deeming it satisfactory. The number of parts tested for integration testing shall initially be small and incrementally rise. The last stage, will be testing the entire system (i.e. system testing).

After system testing succeeds, acceptance testing shall be done by demonstrating the software suite to the Troitsky club. Once they accept our demonstration, our suite is fit for release.

Testing is an iterative process; failures must be corrected and resubmitted. Any tests under the unit testing framework may run automatically. However, all other previous tests must be redone unless the test tests non-critical parts only and that it is painstakingly obvious that the correction did not impact the test.

3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select,

you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

As stated in the Development Plan (see Expected Technology section), unit testing shall occur through an establshed C# unit testing tool, namely MS Test. Code coverage of tests shall be determined by Coverlet, an establshed testing tool. Both are open-source projects of the .NET validation.

Linting shall be done with CS Lint, freely available under the MIT license. See the Development Plan (ibid.).

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

As stated in the SRS (see S.1), our software suite shall contain one main tasks, model analysis. and many auxiliary tasks, namely a materials testing suite, a login suite, a knowledge repository (this includes both file upload and file search), and knowledge transfer tool and a broker to coordinate user tasks.

The main tasks are currently done by hand or through the graphical user interface of computer-aided design software (see SRS G.2). All calculations done by this software project shall be validated through ensuring the answers obtained from the use of this software and the current methods are the same.

Many models will be used in this process and our software and the current methods must yield the same results for each one. To prevent overfitting to our training data, some models shall be hidden during development and debugging and only utilized during testing. The number of models is to be determined, although the number of training models shall not exceed the number of test models and five of each is the current forecasted amount.

The auxiliary tasks shall be validated as follows:

- The **login suite** shall be validated against Troitsky's current list of members and their roles.
- The **knowledge transfer tool** shall be validated against Troitsky's current mechanism for creating transition documents. The club may want to alter the process for the current process is quite informal, and any alterations requests shall be meticulously followed.
- The **knowledge repository**, the **materials testing suite**, and **toolbox broker** are new capabilities which have no benchmark to validate against.

We are lucky to benefit from our primary stakeholder being almost available to us. Frequent consultation and demonstration shall be utilized to gather their feedback. Because they are our client, they will be eager to answer our questions about this project within reason. Before our first release, there shall be an extensive demonstration of our work before the Troitsky club, where we will recieve their feedback and make corrections based on it.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

Type: Functional, Dynamic, Manual, Static etc.

Initial State:

Input:

Output:

How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

Unit tests are an integral part of the verification of the implementation of this project. Below is a sketch of the unit tests that will be employed in this software project.

5.1 Unit Testing Scope

The crown jewel of this project is the model analysis, because all other services offered are pretty simple. The model analysis service contains five (5)

modules: (a) an upload module to upload the STEP file, (b) an external computer-aided design (CAD) module to perform technical engineering calculations, (c) the bridge failure module, (d) the rule compliance module, and (e) the scoring module. The upload module is very straightforward; it can be done with standard file integrity testing. The validation of the calculations of the external CAD module is beyond the software scope of this project; rather, they shall be shown to be accurate by comparing results against the current standards process Troitsky uses and deeming it satisfactory if they are the same for several STEP files, including files purposely hidden from the training data. The last three modules shall be verified through unit testing, having been formalized in the SRS (see S.7).

The other services are the log-in, the materials testing database, the knowledge repository (both adding new files and searching existing files), and the knowledge transfer service to facilitate the creation of transition documents. All these tasks will contain are a login suite, page rendering, role-based access control, CRUD (create, read, update, delete) operations for a database or a shared file store, a form to submit information, and a naive search engine (a simple LIKE-based search is almost certainly sufficient for now). These problems are not novel and be quickly implemented and verified.

Therefore, while unit testing should exist for them, a few will suffice for each. Therefore, the vast majority of unit testing should be directed at the last three sub-modules of the model analysis service.

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

Below is a sketch of automated tests for the functional requirements. First are the three major modules and below them are the auxiliary services.

For the first two tests, two phyla of tests shall be conducted, depressed tests with dummy data, and actual tests with a real STEP file. The former are white box, as the input is artificial and the latter are black box, as the output is based on a bona fide STEP file. Inputting artificial data is the domain of the developers for testing. A real user should input a raw STEP

file and see the output of the system's analysis of it, because calculating and inputting intermediate results an external CAD library would do is highly error-prone.

Due to the straightforwardness of the auxiliary services above, each service shall contain no more than two (2) examples, of unit tests. There may be more in the actual unit testing suite, but this document serves as a sketch, not a complete guide, for the suite.

5.2.1 Bridge Failure Module

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

This module determines whether a bridge fails, i.e., whether there is an element in the bridge whose factor of safety is less than one. The examples below are illustrative of the test types, in reality there will be several for each type.

1. Artificial, Bridge Succeeds

   Type: Automatic

   Initial State: System requests artificial data for an STEP model to analyze.

   Input: This bridge contains two elements. Element e_a is of a material m_a where $\mu_{strength}(m_a) = 5$ and $\sigma_{API}(e_a) = 4$ and Element e_b is of a material m_b where $\mu_{strength}(m_b) = 3$ and $\sigma_{API}(e_b) = 2$.

   Output: $BridgeFailure \equiv false$

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$
\begin{aligned}
BridgeFailure &\equiv (\exists e : \frac{\mu_{\text{strength}}(\text{elementMaterial}(e))}{\sigma_{\text{API}}(e)} < 1) \\
&\equiv \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_a))}{\sigma_{\text{API}}(e_a)} < 1 \ \vee \ \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_b))}{\sigma_{\text{API}}(e_b)} < 1 \\
&\equiv \frac{5}{4} < 1 \ \vee \ \frac{3}{2} < 1 \\
&\equiv \text{false} \vee \text{false} \\
&\equiv \text{false}
\end{aligned}
$$

How test will be performed: Through MsTest within the .NET solution used for implementation

2. Artificial, Bridge Fails

   Type: Automatic

   Initial State: System requests artificial data for an STEP model to analyze.

   Input: This bridge contains two elements. Element e_a is of a material m_a where $\mu_{strength}(m_a) = 5$ and $\sigma_{API}(e_a) = 4$ and Element e_b is of a material m_b where $\mu_{strength}(m_b) = 3$ and $\sigma_{API}(e_b) = 4$.

   Output: $BridgeFailure \equiv true$

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$
\begin{aligned}
BridgeFailure &\equiv (\exists e : \frac{\mu_{\text{strength}}(\text{elementMaterial}(e))}{\sigma_{\text{API}}(e)} < 1) \\
&\equiv \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_a))}{\sigma_{\text{API}}(e_a)} < 1 \ \vee \ \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_b))}{\sigma_{\text{API}}(e_b)} < 1 \\
&\equiv \frac{5}{4} < 1 \ \vee \ \frac{3}{4} < 1 \\
&\equiv \text{false} \vee \text{true} \\
&\equiv \text{true}
\end{aligned}
$$

How test will be performed: Through MsTest within the .NET solution used for implementation

3. Natural

   Type: Automatic

   Initial State: System requests an STEP file that models a bridge

   Input: The STEP file

   Output: If there is an element e where

   $$frac\mu_{\text{strength}}(\text{elementMaterial}(e))\sigma_{\text{API}}(e) < 1$$

   $BridgeFailure \equiv true$. Otherwise, $BridgeFailure \equiv false$.

   Test Case Derivation: From the formalization in (S.7.2 (1)). The external CAD library will return $m_e = elementMaterial(e)$ and $\sigma_{API}(e)$ for each element present in the bridge model. The materials testing module will be invoked to find $\mu_{\text{strength}(m_e)}$. Then, this module will have gained its input to perform its calculations and will yield outputs consistent with the formula provided. For more information, see the above two artificial test cases.

   How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.2 Rules Compliance Module

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

This module determines whether a bridge complies with the rules for the Troitsky competition. Because the competition rules change each year, the fields denoting rule-based constraints are variable so they can be easily updated. The examples below are illustrative of the test types, in reality there will be several for each type.

For brevity, the example rules are the same in all three test cases. The maximum weight is 7 kg, the minimum length is 175 m, the maximum length is 225 m, the minimum width is 45 m, the maximum width is 60 m and the

minimum clearance height is 45 m. Furthermore, let an iB (in bounds) function be defined as

$$iB(val, lower, upper) \equiv lower \leq val \wedge val \leq upper$$

Lastly, variables shall be in the form of $qualQty$ where $qual \in$ (val, min, max) and $Qty \in$ (Wt, Ln, Wd, Ht). The variable shall be understood as "the $qual$ for the Qty". The items in the $qual$ (qualifiers) set are short for value, minimum and maximum respectively, while the items in the Qty (quantities) set are short for weight, length, width and height respectively. If no qualifier is there (e.g. Ln), an implicit qualifier of val shall be assumed.

1. Artificial, Rules Compliant

   Type: Automatic

   Initial State: System requests artificial data for an STEP model to analyze.

   Input: This bridge weighs 5 kg, has a length of 200 m, a width of 50 m and a clearance height of 50 m.

   Output: $isValid \equiv true$

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$\begin{aligned} isValid &\equiv iB(Wt, 0, maxWt) \wedge iB(Ln, minLn, maxLn) \wedge \\ &\quad iB(Wd, minWd, maxWd) \wedge Ht < maxHt \\ &\equiv iB(5, 0, 7) \wedge iB(175, 200, 225) \wedge iB(45, 50, 60) \wedge 45 \leq 50 \\ &\equiv true \wedge true \wedge true \wedge true \\ &\equiv true \end{aligned}$$

   How test will be performed: Through MsTest within the .NET solution used for implementation

2. Artificial, Rules Non-Compliant

Type: Automatic

Initial State: System requests artificial data for an STEP model to analyze.

Input: This bridge weighs 5 kg, has a length of 200 m, a width of 61 m and a clearance height of 50 m.

Output: $isValid \equiv false$

Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$\begin{aligned} isValid &\equiv iB(Wt, 0, maxWt) \wedge iB(Ln, minLn, maxLn) \wedge \\ &\quad iB(Wd, minWd, maxWd) \wedge Ht < maxHt \\ &\equiv iB(5, 0, 7) \wedge iB(175, 200, 225) \wedge iB(45, 61, 60) \wedge 45 \leq 50 \\ &\equiv true \wedge true \wedge false \wedge true \\ &\equiv false \end{aligned}$$

How test will be performed: Through MsTest within the .NET solution used for implementation

3. Natural

   Type: Automatic

   Initial State: System requests an STEP file that models a bridge

   Input: The STEP file

   Output: If,

   $iB(Wt, 0, maxWt) \wedge iB(Ln, minLn, maxLn) \wedge iB(Wd, minWd, maxWd) \wedge Ht < maxHt$

   $isValid \equiv true$. Otherwise, $isValid \equiv false$.

   Test Case Derivation: From the formalization in (S.7.2 (2)). The external CAD library will return the bridge's weight, length width and height. Then, this module will have gained its input to perform its calculations and will yield outputs consistent with the formula provided. For more information, see the above two artificial test cases.

   How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.3 Bridge Scoring Module

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

This module determines the score of a bridge. The score is based on the formula in SRS section S.7.3 (3).

For brevity, there will be two hard constraints, two objectives and two soft constraints. As the real calculations required to determine these values are yet to be decided, a natural unit test would be inappropriate. The score shall not be less than zero. In all three test cases, $w_1 = 5$, $w_2 = 3$, $p_1 = 400$, and $p_2 = 200$. See S.7.3 for the meanings of the abbreviations used here.

1. Violates Hard Constraint

   Type: Automatic

   Initial State: System requests artificial data to run scoring module.

   Input: $r_1 = 0$, $r_2 = 1$, $s_1 = 100$, $s_2 = 50$, $b_1 = 0$, $b_2 = 1$.

   Output: $W = 0$

   Test Case Derivation:

$$\begin{aligned} W &= max(r_1 r_2(w_1 s_1 + w_2 s_2 - (b_1 p_1 + b_2 p_2)), 0) \\ &= max(0(1)(5 * 100 + 3 * 50 - (0 * 400 + 1 * 200)), 0) \\ &= max(0(450), 0) \\ &= 0 \end{aligned}$$

   How test will be performed: Through MsTest within the .NET solution used for implementation

2. Normal

   Type: Automatic

Initial State: System requests artificial data to run scoring module.

Input: $r_1 = 1$, $r_2 = 1$, $s_1 = 100$, $s_2 = 50$, $b_1 = 0$, $b_2 = 1$.

Output: $W = 450$

Test Case Derivation:

$$\begin{aligned} W &= max(r_1 r_2(w_1 s_1 + w_2 s_2 - (b_1 p_1 + b_2 p_2)), 0) \\ &= max(1(1)(5 * 100 + 3 * 50 - (0 * 400 + 1 * 200)), 0) \\ &= max(1(450), 0) \\ &= 450 \end{aligned}$$

How test will be performed: Through MsTest within the .NET solution used for implementation

3. Below Floor of Zero

Type: Automatic

Initial State: System requests artificial data to run scoring module.

Input: $r_1 = 0$, $r_2 = 1$, $s_1 = 50$, $s_2 = 100$, $b_1 = 1$, $b_2 = 1$.

Output: $W = 0$

Test Case Derivation:

$$\begin{aligned} W &= max(r_1 r_2(w_1 s_1 + w_2 s_2 - (b_1 p_1 + b_2 p_2)), 0) \\ &= max(1(1)(5 * 50 + 3 * 100 - (1 * 400 + 1 * 200)), 0) \\ &= max(1(-50), 0) \\ &= 0 \end{aligned}$$

How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.4 Log-in

This module handles an attempt by any person to demonstrate they are the person associated with a given user account. Because single sign-on (SSO) shall be employed, the system shall recieve a confirmation a user is the owner of a specific email address, almost always a McMaster one, not a username and password.

1. Successful Attempt

   Type: Automatic

   Initial State: Session not logged in

   Input: SSO confirmation of an email that matches an active account in system

   Output: Session logged in as that account

   Test Case Derivation:

   - System performs parametrized query on the Users database looking for user accounts that match the inputted confirmation's email
   - System finds one active account in the system that fits the query
   - System logs in session with that active account

   How test will be performed: Through MsTest within the .NET solution used for implementation

2. Failed Attempt

   Type: Automatic

   Initial State: Session not logged in

   Input: SSO confirmation of an email that does not match an active account in system

   Output: Session not logged in, error message shown to user

   Test Case Derivation:

   - System performs parametrized query on the Users database looking for user accounts that match the inputted confirmation's email

- System finds no active accounts that fit the query
- System displays error message to user, session remains not logged in

How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.5 Materials Testing

This module stores the results of materials tests manually done by club members to test the strength of a given material. Any authorized user shall be able to submit a test result and it shall be presumed to be reliable. However an executive user may mark the test unreliable, which will exclude it from all future materials analysis calculations as long as it marked as unreliable.

Automatic tests are a poor fit here because the bulk of the work is to convert a user request into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Results Submission, Group

   Type: Manual, Dynamic

   Initial State: User on materials testing page and may submit a materials test

   Input: An spreadsheet-style document (.csv or .xslx) containing materials testing results (each containing a material and its strength)

   Output: The results are stored as reliable materials testing results. User recieves confirmation of sucessful submission.

   Test Case Derivation:

   - User clicks "Add Materials Test Result" button
   - Graphical user interface (GUI) opens, prompting the upload the spreadsheet-style document
   - User uploads the document and submits it
   - System normalizes the submitted data to obtain values used to add to the materials testing database

- System adds new rows to the database containg the obtained values and the reliable flag set to true
- System recieves confiramation from the database that the row was added successfully
- User recieves confirmation from the system the materials test was submitted successfully

How test will be performed: Manually, through the front end.

2. Test Marked as Unreliable

   Type: Manual

   Initial State: User on materials testing page and may deem a test unreliable

   Input: The test a user deems unreliable

   Output: The test is marked unreliable; user recieves corresponding confirmation

   Test Case Derivation:

   - User clicks "Mark Test Unreliable" button
   - System displays list of materials tests submitted, in an order descending by date.
   - User picks the test they have judged to be unreliable from the menu.
   - The user clicks the "Mark Unreliable" button.
   - A confirm dialog opens, and the user selects the "Yes, I am sure" button. The dialog closes.
   - The system submits a query to the materials testing database to mark the picked test unreliable.
   - The database marks that test unreliable and sends a confirmation to the system.
   - The system tells the user through an on-screen message that the test (with a brief description of it) was successfully marked unreliable.

   How test will be performed: Manually, through the front-end

5.2.6 Knowledge Repository

This module contains the club-wide document share and lets users add documents to it, alter or delete their own documents or any document as an executive, and search, preview and download documents. There is a role-based access system based on account classification, so not all users may see every document added to the document share.

Automatic tests are a poor fit here because the bulk of the work is to convert a user request into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Alter Document

   Type: Manual, Dynamic

   Initial State: User is logged in and on the knowledge repository page. They also may perform their desired action.

   Input: The document name, the action, and the contents if the action is not delete.

   Output: The action is done to the document with the inputted name, adding the contents to the document if the action was not delete. Test Case Derivation:

   - User clicks one of the following buttons: (a) "Add Document", (b) "Replace Document", (c) "Edit Document" or (d) "Delete Document.
   - If the button clicked is (a) or (b):
     - A dialog opens, with an empty markdown textbox and a button to open an upload wizard.
     - The user uploads a document from the upload wizard.
     - Alternatively, the user adds to the markdown textbox.
     - The user clicks the corresponding submit button, submitting their desired input to the system.
   - If the button clicked is (c):
     - A dialog opens with a markdown textbox containing the existing markdown document.

- The user edits the document to their heart's desire.
- The user submits the document.

- If the user clicks button (d), the request is submitted directly.
- The system request the drive fulfill the request. The drive does and confirms it to the system.
- The system displays a confirmation message to the user.

How test will be performed: Manually, through the front end.

2. Obtain Document

   Type: Manual

   Initial State: User on knowledge repository page and access a document

   Input: The document name

   Output: The user gains access to document, with a preview (unlessit cannot be done) and a download option

   Test Case Derivation:

   - User clicks "Search for Document" button
   - User enters suitable query to search bar.
   - User hits "Search" and results are returned in declining likeness to the query.
   - The user selects their desired document.
   - A new page opens with the preview. The user is also presented with a button they may click if they wish to download the document.

   How test will be performed: Manually, through the front-end

5.2.7 Knowledge Transfer

This module tracks whether a user may leave the club soon. If so, the module will give them an alert on their user homepage, prompting them to fill out their transition document. The module then takes input through a form. Once the user submits it, the system adds it to the knowledge repository.

Automatic tests are a poor fit here because the bulk of the work is to convert user-entered text into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Submit Transfer Document

   Type: Manual, Dynamic

   Initial State: User is logged in and may leave the club soon

   Input: The text entered into each input field of the tranfer form

   Output: The text is converted into one easy-to-read document, which is added to the knowledge repository.

   - User finds alert "Make Transition Document" on user homepage.
   - User clicks the alert, opening up the transition document form
   - The user fills out the form and submits it
   - The system converts it into an easily readable and understandable document
   - The system uploads the document to the knowledge repository.
   - Once recieving a confirmation of success, it displays a "form successfully submitted" message to the user

   How test will be performed: Manually, through the front end.

5.2.8 Toolbox Broker

This module fashions and displays the homepage. It furthermore transfers the user from the home page to the pages for every other module they can access, besides the public logged out module.

Automatic tests are a poor fit here because the bulk of the work is to convert a user request into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Go to Service

   Type: Manual, Dynamic

   Initial State: User is logged in and is on home page. They may access the page they desire to access.

   Input: The user clicks the buttton corresponding to their desired action.

   Output: The system displays the page for their desired action.

   - The user clicks the button
   - The system opens up the corresponding page

   How test will be performed: Manually, through the front end.

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

Most information to satisfy the non-functional requirements come from the functional tests above. Timing requirements shall be handled by time-keeping and recording a fail if the test goes overtime, even if it the desired result is ultimately obtained. Logging will be a byproduct of the functional tests, and some requirements, like weekly backup, cannot be effectively unit tested. One additional test is required, and it is detailed below.

5.3.1 Module Analysis

Unlike the counterpart subsections in the functional unit section, this is tested as one unit because the user will evaluate all its submodules in one action.

1. Timing

   Type: Manual, Dynamic

Initial State: User on model analysis page

Input/Condition: The STEP file of a bridge model.

Output/Result: Whether the bridge will fail, whether the bridge is rules compliant and its score.

How test will be performed: Manual, Dynamic

5.4 Traceability Between Test Cases and Modules

See the subsection foreword above. The crown jewel of this service, model analysis, is throughly tested; upload through the sole non-functional unit test, the three calculation sub-mdoules are formalized and tested through their formalizations, while the external CAD software library's accuracy is assumed. For each auxiliary module listed above, which contains every service this system calls for, there is at least one functional unit test case.

References

Author Author. System requirements specification. https://github.com/..., 2019.

Pedro Azevedo, Mason Azzopardi, Michael Roberts, and Jack Easton. Hazard Analysis, Troitsky Toolbox. Team 14, SFWRENG 4G06 Capstone, McMaster University, 2026a. URL https://github.com/JackEaston4/Troitsky-Toolbox/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf.

Pedro Azevedo, Mason Azzopardi, Michael Roberts, and Jack Easton. Software Requirements Specification, Troitsky Toolbox. Team 14, SFWRENG 4G06 Capstone, McMaster University, 2026b. URL https://github.com/JackEaston4/Troitsky-Toolbox/blob/main/pdfs/SRS/SRS.pdf.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?